



**ADRIELLY FERREIRA DA SILVA
MARCO ANTONIO ROQUINI ESPUDARIO
MARIA CLARA SOUZA ROSA
MILENA DE LOURDES BARBOSA**

GCC129 – Documentação Técnica e Engenharia de Software

LAVRAS – MG

2026

1. Introdução e Contextualização do Problema

1.1 O Problema Real

No cenário de alta densidade de demandas informacionais e acadêmicas, os usuários enfrentam severas barreiras para gerir suas rotinas devido à fragmentação de dados em múltiplas ferramentas desconectadas. Aplicativos de calendário e gerenciadores de listas de tarefas operam de forma isolada, atuando como repositórios estáticos e passivos de informação.

Como consequência, observa-se o acúmulo de tarefas sem critérios objetivos de relevância, o surgimento de conflitos crônicos de horários nas agendas, o esquecimento de prazos fatais e uma perda acentuada de produtividade decorrente do esforço em organizar a própria organização. Essas ferramentas tradicionais falham por não prover suporte cognitivo contextualizado ou inteligente para mitigar a fadiga de decisão do usuário.

1.2 Público-Alvo e Justificativa

O sistema foi projetado para atender:

- **Estudantes Universitários:** Sob alta carga de avaliações, horários de aulas dinâmicos e prazos de projetos distribuídos.
- **Profissionais e Gestores Corporativos:** Que necessitam gerenciar múltiplos fluxos de trabalho e reuniões concorrentes.
- **Equipes de Trabalho:** Que demandam uma triagem rápida de entregáveis e alocação eficiente de tempo disponível.

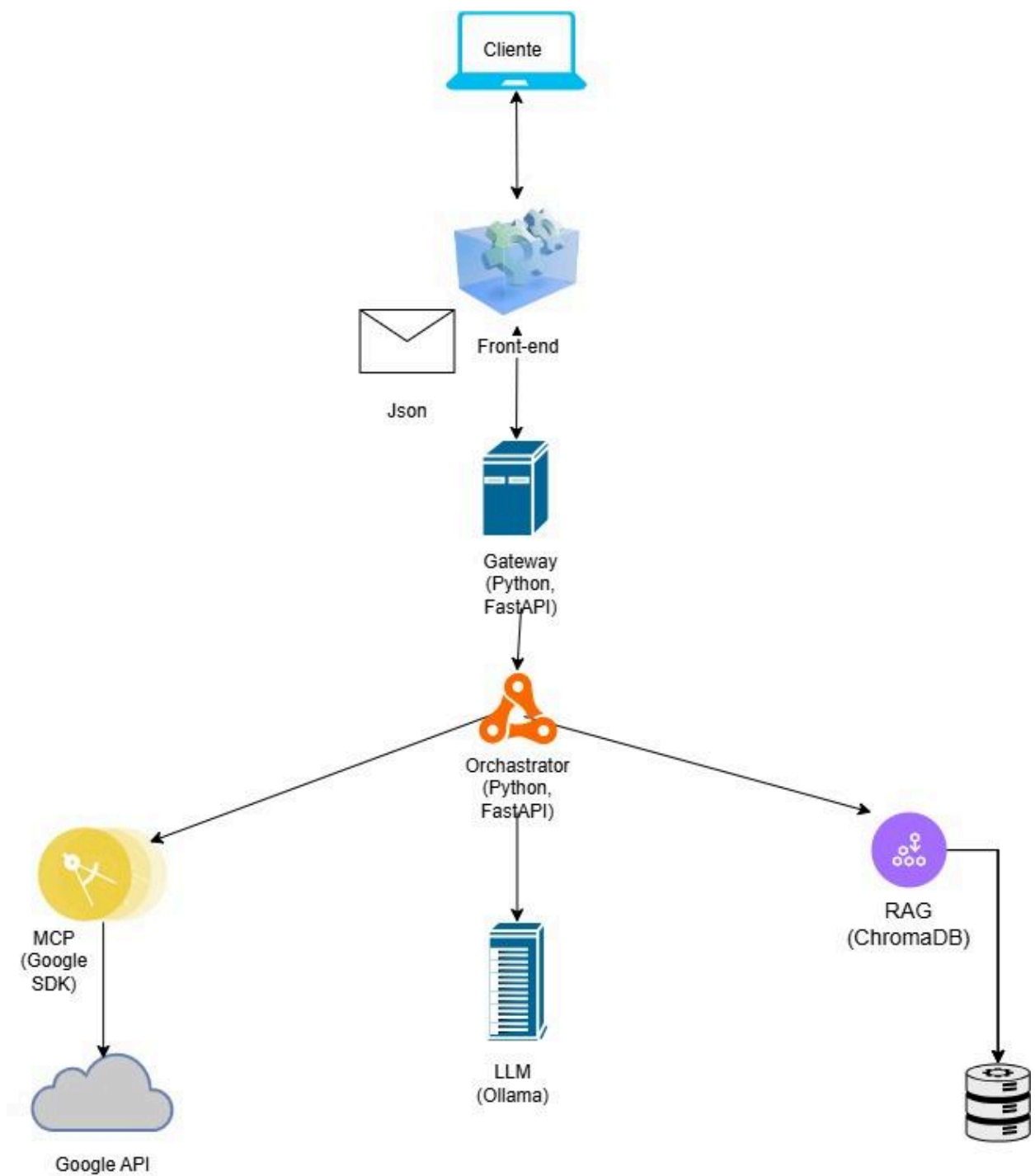
1.3 Impacto da Solução

Ao unificar IA Generativa, recuperação semântica e integração de ecossistemas externos em uma malha distribuída, a solução promove:

- **Decisão Inteligente Baseada em Contexto:** A IA avalia dinamicamente o panorama do dia do usuário para sugerir ações práticas imediatas.
- **Redução de Sobrecarga Cognitiva:** Automatização da análise de urgência e importância com base em dados em tempo real.
- **Otimização do Tempo Útil:** Bloqueios de tempo dinâmicos e resolução automática de choques de horários.

2. Visão Geral do Sistema Distribuído

O ecossistema é baseado em uma **arquitetura de microsserviços totalmente desacoplados e síncronos**, onde cada componente executa um papel especialista e se comunica exclusivamente por meio de chamadas HTTP com payloads estruturados em formato JSON.



2.1 Matriz de Topologia de Rede e Portas

Microsserviço	Porta Local	Tecnologia Base	Descrição Funcional
API Gateway	8004	Python, FastAPI	Porta de entrada, controle de acesso e tratamento de CORS.

Orchestrator Service	8000	Python, FastAPI	Cérebro do sistema. Valida intenção e coordena o fluxo distribuído.
RAG Service	8001	ChromaDB	Recuperação semântica de metodologias científicas de produtividade.
MCP Service	8002	Google SDK	Conector padronizado com as APIs oficiais do Google Tasks/Calendar.
LLM Service	8003	Ollama / Llama 3	Motor de inferência e geração cognitiva contextualizada.

3. Detalhamento dos Componentes e Implementação Código

3.1 API Gateway ([gateway/main.py](#))

Atua na borda (*edge*) da infraestrutura distribuída. É o componente responsável por receber o tráfego do cliente externo e encaminhá-lo de maneira segura para a malha interna de serviços.

- **Decisões de Engenharia:** Utiliza o cliente assíncrono [httpx.AsyncClient](#) para garantir chamadas não-bloqueantes de I/O, e define um limite rígido de tolerância de rede (`timeout=180.0`) para suportar o tempo de processamento elástico demandado pelo modelo de linguagem.

3.2 Orchestrator Service ([orchestrator/main.py](#))

O coração algorítmico do sistema distribuído. Ele não gerencia conexões de rede externas de usuários, focando unicamente na coordenação dos microsserviços internos.

- **O Padrão Porteiro (Guardrail de Intenção):** O orquestrador realiza uma pré-triagem cognitiva da requisição submetida pelo usuário enviando-a ao [LLM Service](#). Se a pergunta for classificada como fora do domínio do sistema (ex: perguntas sobre clima ou curiosidades gerais), o fluxo é interceptado imediatamente com retorno `0`, poupando chamadas de rede e custos operacionais ao banco vetorial e às ferramentas externas do MCP.
- **Injeção de Metadados Cronológicos:** O orquestrador captura o momento exato do sistema via `datetime.now()` e injeta no prompt mestre para garantir que o modelo Llama 3 processe de forma precisa as janelas de tempo fornecidas pela agenda do Google.

3.3 RAG Service ([rag_service/rag.py](#) & [main.py](#))

Componente responsável por injetar conhecimento especializado em tempo real nas requisições, eliminando alucinações e vieses genéricos do modelo de linguagem.

- **Mecanismo de Bootstrap Autônomo:** No momento em que o microserviço é carregado na rede, ele executa uma rotina de verificação defensiva. Caso a coleção vetorial "regras" não esteja presente no banco de dados **ChromaDB**, o próprio serviço instancia a coleção e realiza a inserção (*seeding*) automatizada dos preceitos conceituais da Matriz de Eisenhower.

3.4 MCP Service (`mcp_service/tools.py` & `main.py`)

O **MCP Service** é o responsável direto por estender as capacidades do modelo de linguagem (LLM), agindo como uma ponte de ferramentas padronizada e segura entre o Orquestrador e o ecossistema do Google. Ele implementa o protocolo de contexto por meio de duas APIs críticas estruturadas sob autenticação OAuth2 unificada:

A. Integração Explícita com o Google Tasks API

Este componente é projetado para ler de forma estruturada a fila de afazeres do usuário armazenada em nuvem.

- **Inicialização e Autenticação:** Constrói o cliente de serviço utilizando a biblioteca de descoberta do Google (`googleapiclient.discovery.build`) especificando o serviço 'tasks' e a versão 'v1' com as credenciais previamente validadas.
- **Execução da Requisição:** Invoca o método `service.tasks().list(tasklist='@default').execute()` para extrair dados da lista de tarefas padrão (*default*) do perfil autenticado do usuário.
- **Mapeamento e Extração de Domínio (ACL):** O microserviço intercepta o dicionário massivo gerado pela resposta HTTP do Google e extrai apenas a informação de interesse, que são os títulos das tarefas:

None

- ```
items = results.get('items', [])
```
- ```
return [task['title'] for task in items] if items else ["Nenhuma tarefa encontrada."]
```

- Isso simplifica drasticamente o payload trafegado de volta na rede interna e protege a privacidade das IDs internas de banco do Google.

B. Integração Explícita com o Google Calendar API

Este componente é projetado para atuar na análise de restrições rígidas temporais na rotina diária do usuário.

- **Inicialização e Autenticação:** Instancia o construtor do cliente do Google Calendar definindo o serviço como 'calendar' na versão 'v3'.
- **Tratamento e Restrições Temporais de Rede:** Para evitar que o modelo processe compromissos que já aconteceram no passado, a integração captura o horário absoluto de execução e o converte para o formato ISO 8601 estipulado pela API do Google:

None

- ```
from datetime import datetime
```
- ```
now = datetime.utcnow().isoformat() + 'Z' # 'Z' indica fuso horário UTC (Zulu)
```

- **Parâmetros de Consulta Otimizados:** Dispara a consulta `service.events().list(...)` refinando a chamada com parâmetros específicos que protegem o desempenho do sistema distribuído:
 - `calendarId='primary'`: Restringe a busca ao calendário principal do usuário.
 - `timeMin=now`: Configura o filtro temporal para retornar apenas eventos futuros ou em andamento.
 - `maxResults=10`: Limita o tamanho do array a no máximo 10 eventos, evitando estouro de janela de contexto do LLM.
 - `singleEvents=True`: Expande eventos recorrentes complexos em instâncias únicas (essencial para que compromissos repetitivos diários ou semanais sejam mapeados como blocos de tempo individuais e legíveis).
 - `orderBy='startTime'`: Força a API do Google a ordenar a resposta de forma cronológica antes do tráfego de rede local.
- **Formatação de Dados:** Converte o payload de datas complexo do calendário em frases diretas e simplificadas de texto legível por máquina:

None

- ```
return [f"{event['summary']} às {event['start'].get('dateTime', event['start'].get('date'))}" for event in events]
```

- Dessa forma, o orquestrador e a IA recebem uma representação literal exata do dia (ex: `"Reunião de SD às 2026-06-16T14:00:00-03:00"`), permitindo um cálculo de impacto de tempo preciso sem precisar computar fusos horários complexos em tempo real.

### 3.5 LLM Service (`llm_service/llm.py` & `main.py`)

Nó de computação focado em inferência inteligível de linguagem. Abstrai o servidor local do Ollama exposto na máquina hospedeira.

- **Consistência Lógica:** Fixa o parâmetro estatístico `temperature` em `0.0`. Essa configuração força a rede neural a atuar de forma puramente determinística, garantindo previsibilidade e repetibilidade no algoritmo final de ordenamento de tarefas diárias.

## 4. Padrões de Projeto e Arquitetura Aplicados

### 4.1 Gateway Routing (Roteamento de Gateway)

O sistema oculta completamente sua topologia interna do consumidor final por meio do padrão *Gateway Routing*. O Frontend conhece exclusivamente o endereço IP e a porta 8004 do API Gateway. Caso um microserviço interno mude de porta, servidor ou linguagem de programação, o cliente externo permanece inalterado, garantindo um acentuado nível de transparência de localização em sistemas distribuídos.

#### 4.2 Anti-Corruption Layer (Camada Anticorrupção - ACL)

As respostas nativas fornecidas pelo SDK do Google (Calendar e Tasks) retornam metadados volumosos, aninhados e altamente proprietários. O `mcp_service/tools.py` atua estritamente como uma **Camada Anticorrupção (ACL)**. Ele traduz o objeto complexo externo em coleções primitivas simplificadas (ex: listas de títulos de tarefas e strings textuais contendo horários limpos). Isso evita que o modelo interno do Orquestrador sofra acoplamento com contratos de dados de terceiros.

#### 4.3 Padrão Impulsionado por Eventos de Ciclo de Vida (Startup Hooks)

Para evitar falhas em cascata por *timeouts* devido a fluxos de autenticação humana em tempo de execução, os microserviços MCP e RAG utilizam tratamento assíncrono antecipado na inicialização da rede para garantir o estado ideal de suas coleções e tokens antes do recebimento de requisições de produção.

### 5. Recuperação de Informação (RAG) e Inteligência Artificial

#### 5.1 O Mecanismo RAG

O RAG baseia-se na vetorização de documentos textuais que ditam as melhores práticas científicas de produtividade corporativa e pessoal. Ao receber a string da requisição do usuário através do Orquestrador, o ChromaDB executa internamente a conversão da entrada em um vetor numérico de embeddings e realiza o cálculo de distância vetorial em relação à base de conhecimento indexada.

A recuperação utiliza a métrica de similaridade por cosseno para identificar quais preceitos teóricos são os mais adequados para responder ao contexto do usuário:

$$\text{Semelhança}(q, d_i) = (q \cdot d_i) / (|q| \cdot |d_i|)$$

O microserviço foi parametrizado com `n_results=2`, garantindo que apenas as duas fatias de conhecimento mais relevantes semanticamente sejam injetadas no prompt, otimizando o consumo de memória do modelo generativo.

#### 5.2 Engenharia de Prompt e Regras Axiomáticas de Priorização

A inteligência do sistema é moldada por diretrizes de inferência inflexíveis definidas no Orquestrador. O prompt força o modelo a interpretar as variáveis de dados sob os seguintes pesos hierárquicos:

1. **Restrições Absolutas (Calendário - MCP):** Eventos do Google Calendar são interpretados como janelas de tempo imutáveis e fixas. Nenhuma tarefa pode ser agendada por cima de reuniões ou compromissos existentes.
2. **Prioridade Nível 1 (Saúde e Bem-estar):** Atividades ligadas a refeições, sono e saúde física possuem precedência imediata sobre metas corporativas.
3. **Prioridade Nível 2 (Compromissos Acadêmicos e Profissionais):** Tarefas do Google Tasks correlacionadas a prazos de entregáveis iminentes.

4. **Acoplamento Metodológico (RAG):** Mapeamento automático de tarefas dentro dos quadrantes urgência/importância da Matriz de Eisenhower recuperada pelo banco vetorial ChromaDB.

## 6. Fluxo de Dados de uma Requisição End-to-End

O comportamento operacional do ecossistema distribuído durante o ciclo completo de uma consulta segue a ordem cronológica descrita abaixo:

**Ativação:** O usuário submete no Frontend a frase: *"O que eu preciso fazer hoje à tarde?"*.

1. **Interceptação de Borda:** O **API Gateway (8004)** recebe a chamada HTTP, valida as diretrizes de CORS e encaminha a requisição via rede interna para o endpoint do **Orchestrator Service (8000)**.
2. **Classificação de Intenção (Guardrail):** O Orquestrador invoca o **LLM Service (8003)** repassando apenas a pergunta. O Llama 3 analisa a string e retorna estritamente o caractere "1" (confirmando pertinência ao domínio de tarefas).
3. **Extração de Contexto Distribuído:** O Orquestrador dispara duas chamadas síncronas simultâneas:
  - Uma para o **RAG Service (8001)** via endpoint `/context`, obtendo as regras da Matriz de Eisenhower mais próximas à pergunta.
  - Uma para o **MCP Service (8002)** via endpoints `/tasks` e `/calendar`, capturando em tempo real a lista de tarefas reais pendentes no Google Tasks e os eventos agendados no Google Calendar do usuário.
4. **Injeção e Fusão:** O Orquestrador compila os retornos obtidos, unifica os dados temporais do sistema com as metodologias de produtividade e as pendências em um super-prompt de contexto.
5. **Cognição:** O prompt robusto é submetido ao **LLM Service (8003)** no endpoint `/generate`. O modelo de linguagem computa o plano diário estruturado em formato Markdown.
6. **Desempilhamento de Resposta:** O Orquestrador recebe a string textual do LLM, repassa-a para o API Gateway que, por sua vez, serializa a resposta final em formato JSON seguro e a entrega ao Frontend para renderização visual ao usuário.

## 7. Instruções para Instalação, Configuração e Execução

### 7.1 Requisitos de Infraestrutura Local

- Interpretador Python 3.11 ou superior instalado.
- Serviço Ollama ativo localmente com o modelo `llama3` baixado via terminal (`ollama pull llama3`).
- Arquivo `credentials.json` gerado no painel do Google Cloud Console com permissões para as APIs do Google Tasks e Google Calendar, posicionado na raiz da pasta `mcp_service/`.

### 7.2 Instalação de Dependências Coletivas

Navegando a partir do diretório raiz do projeto unificado, execute a instalação dos pacotes listados no arquivo `requirements.txt`:

Bash



None

```
pip install fastapi uvicorn chromadb requests httpx google-auth google-auth-oauthlib
google-api-python-client
```

### 7.3 Scripts de Inicialização Automatizada da Malha Distribuída

Para simplificar a inicialização coordenada de todos os microsserviços concorrentes na máquina de testes, foram desenvolvidos scripts de lote automatizados que levantam a arquitetura em paralelo.

#### Script para Ambientes Windows (**start\_all.bat**)

DOS

None

```
@echo off
echo Inicializando a Malha de Microsservicos - Gamma
echo =====

start "Orchestrator Service - 8000" cmd /k "cd orchestrator && uvicorn main:app --port 8000"
start "RAG Service - 8001" cmd /k "cd rag_service && uvicorn main:app --port 8001"
start "MCP Service - 8002" cmd /k "cd mcp_service && uvicorn main:app --port 8002"
start "LLM Service - 8003" cmd /k "cd llm_service && uvicorn main:app --port 8003"
start "API Gateway Borda - 8004" cmd /k "cd gateway && uvicorn main:app --port 8004"

echo =====
echo Todos os microsservicos foram disparados com sucesso!
```

#### Script para Ambientes Linux / macOS (**start\_all.sh**)

Bash

None

```
#!/bin/bash
echo "Inicializando a Malha de Microsservicos - Gamma"
echo "===== "

uvicorn orchestrator.main:app --port 8000 &
uvicorn rag_service.main:app --port 8001 &
uvicorn mcp_service.main:app --port 8002 &
uvicorn llm_service.main:app --port 8003 &
uvicorn gateway.main:app --port 8004 &

echo "===== "
echo "Todos os nos da rede distribuida estao rodando em background!"
wait
```

Para validar a integridade completa da arquitetura distribuída operando de ponta a ponta, pode-se realizar uma chamada utilizando ferramentas de teste HTTP (Postman/Curl) ou diretamente pelo navegador no endereço exposto pelo nó de borda:

<http://127.0.0.1:8004/priorizar?pergunta=Quais sao as minhas tarefas críticas para hoje?>

